BD AI Engineer — Take-Home Case Study

Welcome

Thank you for progressing to the technical case study stage for the **AI Engineer** role. This exercise is designed to let you showcase how you think, design, build, and ship a real AI system — the way you would on the job.

We value **practical engineering, sound architecture, and clear communication** over flashy demos. Please treat this as a representative slice of work, not a perfection contest.

Time allocation	4–5 working days
Choose	**ONE** of the two options below
Tech constraint	**Open-source tools & models only** (the demo must run without paid API keys)
Deliverables	Public GitHub repo + working demo (live)
Demo format	5–10 min live walkthrough in the interview, **or** a recorded video if not available

You may optionally add a paid-API fallback, but the **core system must be fully runnable with open-source components**.

Choose ONE Option

Option 1 — Multi-Format Document/File Chat

("A Copilot / ChatGPT-Killer")

Objective

Build a **robust, multi-agent document chat system** that lets a user upload many different file types and have an intelligent, grounded conversation across them — accurate, fast, and trustworthy enough to compete with Copilot / ChatGPT file chat.

Scope — "Multi-Format"

Your system should ingest and reason over a mix of:

- **Documents:** PDF, DOCX, TXT, Markdown, PPTX
- **Spreadsheets/Data:** CSV, XLSX (real structured Q&A, not just text dumps)
- **Images:** OCR + visual understanding (scanned docs, charts, screenshots)
- **Code files:** read & explain code
- *(Bonus)* Audio/video transcripts, HTML/web pages

Why Multi-Agent

Rather than a single monolithic prompt, design **specialized agents** working together, for example:

- **Router/Planner Agent** — routes the query to the right agent(s)
- **Retrieval Agent** — hybrid (semantic + keyword) search over a vector store
- **Table/Data Agent** — structured spreadsheet reasoning (text-to-SQL / pandas)
- **Vision/OCR Agent** — extracts & interprets images and charts
- **Synthesis Agent** — combines results into one grounded answer
- **Citation/Verification Agent** — attaches sources & checks for hallucinations

Engineering Challenges to Demonstrate

- Layout-aware **chunking & parsing** (tables, headers, structure)
- **Hybrid retrieval + reranking** for accuracy

- **Citations / source attribution** (file name, page number)
- **Cross-document** reasoning
- **Conversation memory** across turns
- Handling **large files** and **many files** at once
- Graceful **"I don't know"** behavior

Suggested Open-Source Stack

- **Orchestration:** LangGraph, LlamaIndex, CrewAI, AutoGen
- **LLMs:** Llama 3.x, Mistral, Qwen, Phi (via Ollama / vLLM / HuggingFace)
- **Embeddings:** BGE, E5, GTE, nomic-embed
- **Vector DB:** Qdrant, Chroma, Weaviate, Milvus
- **Parsing:** Unstructured, PyMuPDF, Docling, Marker, Tesseract / PaddleOCR
- **UI:** Streamlit, Gradio, Chainlit

What Would Impress Us

- Faster or more accurate citations than the incumbents
- Superior table/spreadsheet reasoning
- Confidence scoring or self-verification
- Fully **local / offline** operation (privacy advantage)

Option 2 — Autonomous Data Science Multi-Agent System

("Build Your Own Data Science Agent")

Objective

Build a **multi-agent system that performs an end-to-end data science workflow** from a raw dataset — with minimal human intervention — producing a trained model and a clear report.

Scope — End-to-End Pipeline

1. **Data Understanding** — auto EDA, schema/type detection, target inference, data-quality report
2. **Preprocessing** — missing values, outliers, encoding, scaling, leakage checks

3. **Feature Engineering** — interactions, transformations, selection, datetime/text/categorical handling
4. **Modeling** — algorithm selection, training, hyperparameter tuning, cross-validation
5. **Post-Processing** — metrics, confusion matrix, ROC, explainability (SHAP), error analysis
6. **Reporting** — auto-generated summary of decisions, results, and recommendations

Why Multi-Agent

Mirror a real data science team with specialized agents:

- **EDA Agent** — profiles the data
- **Preprocessing Agent** — cleans & transforms
- **Feature Engineering Agent** — creates & selects features
- **Modeling Agent** — trains & tunes models
- **Evaluation/Critic Agent** — judges results, triggers retries if poor
- **Orchestrator** — manages pipeline, state, and agent handoffs

A strong submission shows an **agentic feedback loop** (e.g., the Critic agent asks the Feature agent to retry when metrics are weak).

Engineering Challenges to Demonstrate

- **Generalization** — works on a *new, unseen* dataset (not hardcoded)
- **Both classification & regression** support
- **Code generation + safe execution** (agents writing/running pandas/sklearn)
- **Reasoning transparency** — why each decision was made
- **Robustness** — handles messy data without crashing
- **Reproducibility** — saved artifacts, configs, logs

Suggested Open-Source Stack

- **Orchestration:** LangGraph, AutoGen, CrewAI
- **LLMs:** Llama 3.x, Qwen2.5-Coder, Mistral (via Ollama / vLLM)
- **ML Libraries:** scikit-learn, XGBoost/LightGBM, pandas, Optuna

- **EDA/Explainability:** ydata-profiling, SHAP, matplotlib/plotly
- **Code Execution:** restricted Python sandbox / e2b / Jupyter kernel
- **UI:** Streamlit, Gradio

What Would Impress Us

- Self-correcting loop that **improves the score across iterations**
- Clean, readable auto-generated report
- Works on a **dataset we hand you live** during the demo

Evaluation Criteria (Both Options)

Area	What We Look For
Architecture	Clear multi-agent design, sensible orchestration, modularity
Robustness	Handles edge cases, large/messy inputs, fails gracefully
Code Quality	Readable, structured, documented, typed
Demo	Actually runs; clear walkthrough of capabilities
README	Setup steps, architecture diagram, design decisions, trade-offs
Innovation	What makes it better than the obvious baseline
Open Source	Fully reproducible without paid keys

Required Deliverables

1. **Public GitHub repository** with a clean commit history
2. **README** including:
    - Setup / run instructions
    - Architecture diagram
    - Key design decisions & trade-offs
    - Known limitations & future work
3. **Demo:** live during the interview **or** a 5–10 min recorded video
4. **(Bonus)** Dockerfile / one-command setup

Ground Rules & Tips

- **Scope smartly.** We'd rather see a smaller system that is robust and well-explained than a broad one that is fragile. Document what you'd build with more time.

- **Show your thinking.** Comments, commit messages, and the README all count. Trade-offs and "what I'd do next" are valued highly.
- **Use of AI tools** (Copilot, ChatGPT, Cursor) is allowed and encouraged — but you must fully understand and be able to explain every part of your submission.
- **Keep it reproducible.** If we can't run it, we can't fully evaluate it.
- **Ask questions.** If anything is unclear, reach out — knowing when to ask is a strength.

Submission

Please reply to this email with:

- 🔗 Your **GitHub repo link**
- 📽️ Your **demo video link** (if not presenting live)
- 📝 Any notes on assumptions or setup

We're genuinely excited to see what you build. Good luck — and have fun with it! 🚀